Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Катышев М.М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 07.11.26

Москва, 2026

Постановка задачи

Вариант 9.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. Дочерний процесс производит деление первого числа команда, на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы и функции:

- read(fd, buf, count) — читает до count байт из дескриптора fd в буфер buf. Возвращает число прочитанных байт, 0 — конец файла, -1 — ошибка. Используется в родительском процессе для чтения имени файла из stdin и самого файла с командами.
- write(fd, buf, count) — пишет до count байт из буфера buf в дескриптор fd. Возвращает число записанных байт, -1 — ошибка. Используется для вывода результатов в stdout и сообщений об ошибках в stderr.
- open(path, flags) — открывает файл по пути path с флагами flags (в работе — O_RDONLY, только чтение). Возвращает файловый дескриптор, -1 — ошибка.
- close(fd) — закрывает файловый дескриптор fd, освобождая ресурс. Используется после завершения работы с файлами, памятью и семафорами.
- fork() — создаёт дочерний процесс как копию текущего. Возвращает: родителю — PID дочернего процесса (> 0), дочернему — 0, при ошибке — -1.
- shm_open(name, flags, mode) — создаёт или открывает именованный объект разделяемой памяти (POSIX shared memory) с заданным именем. Возвращает файловый дескриптор, -1 — ошибка. Используется для выделения области памяти в оперативной системе (а не на диске), к которой будут иметь доступ оба процесса.
- ftruncate(fd, length) — устанавливает размер объекта разделяемой памяти в length байт. Необходимо вызвать, так как по умолчанию объект создаётся размером 0.
- mmap(addr, length, prot, flags, fd, offset) — отображает объект (в данном случае разделяемую память) в адресное пространство процесса. Возвращает указатель на отображённую память, что позволяет работать с ней как с обычным массивом или структурой. Флаг MAP_SHARED гарантирует, что изменения будут видны всем процессам.
- munmap(addr, length) — удаляет отображение памяти из адресного пространства процесса.

- shm_unlink(name) — удаляет именованный объект разделяемой памяти из системы. Необходим для очистки ресурсов ядра после завершения работы программы.
- sem_open(name, flags, mode, value) — создаёт или открывает именованный семафор с начальным значением value. Используется для синхронизации процессов.
- sem_wait(sem) — блокирует выполнение потока, если значение семафора равно 0. Если значение больше 0, оно уменьшается на 1, и выполнение продолжается.
- sem_post(sem) — увеличивает значение семафора на 1. Если на этом семафоре ожидали другие процессы (были заблокированы в sem_wait), один из них будет разблокирован.
- sem_close(sem) и sem_unlink(name) — закрывают дескриптор семафора и удаляют его из системы соответственно.
- waitpid(pid, &status, options) — ожидает завершения дочернего процесса с указанным pid и записывает статус в status. Используется родителем для синхронизации и получения кода возврата ребёнка.

В рамках лабораторной работы было разработано две программы (или один процесс, разделяющийся через fork). В отличие от предыдущих работ, взаимодействие между процессами осуществляется не через каналы (pipe), а через именованную разделяемую память (shared memory) и именованные семафоры.

На начальном этапе родительский процесс читает имя файла из стандартного потока ввода системным вызовом read, открывает файл и считывает из него все данные. Считанные данные разбираются на числа и сохраняются в структуру SharedData, которая будет размещена в разделяемой памяти. Для этого родитель генерирует уникальные имена объектов на основе своего PID (например, /shm_12345), создаёт объект через shm_open, задаёт ему размер через ftruncate и отображает в своё адресное пространство с помощью mmap. Аналогичным образом создаются два именованных семафора с начальным значением 0: sem_data (сигнализирует о готовности данных) и sem_result (сигнализирует о готовности результата).

После подготовки памяти и заполнения её распарсенными данными вызывается fork(), порождающий дочерний процесс. Дочерний процесс сразу блокируется на sem_wait(sem_data), ожидая, пока родитель подаст сигнал о готовности данных. Родитель вызывает sem_post(sem_data), после чего дочерний процесс получает доступ к массивам чисел из разделяемой памяти, производит их деление (проверяя делители на ноль) и записывает итоговые значения в массив output_result. По завершении расчётов дочерний процесс вызывает sem_post(sem_result), сигнализируя родителю о готовности результатов, и завершает работу через exit(0). Если при делении встречается ноль, дочерний процесс устанавливает флаг error_flag в структуре памяти, будит родителя через sem_post(sem_result) и завершается с кодом 1.

Родительский процесс после sem_post(sem_data) ожидает сигнала на семафоре sem_result. Проснувшись, он ожидает завершения дочернего процесса через waitpid и проверяет флаг error_flag. Если ошибок не было, родитель выводит результаты из разделяемой памяти в стандартный поток вывода. В противном случае выводится сообщение об ошибке деления на ноль. В завершение работы родительский процесс освобождает все выделенные ресурсы: отключает память через munmap, закрывает дескрипторы и удаляет именованные объекты из системы с помощью shm_unlink и sem_unlink.

Код программы

lab.cpp

```
#include <unistd.h>
#include <fcntl.h>
#include <cstring>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>
#include <cstdio>
#include <cstdlib>

#define MAX_NUMS 10000
#define MAX_LINES 1000

struct SharedData {
    double input_nums[MAX_NUMS];
    int line_lengths[MAX_LINES];
    int total_lines;

    double output_result[MAX_LINES];
    int error_flag;
};

int parse_line(char* line, double* nums) {
    int count = 0;
    char* p = line;
    char* end;
    while (true) {
        double x = strtod(p, &end);
        if (end == p) break;
        nums[count++] = x;
        p = end;
    }
    return count;
}

int main() {
    char file_name[256];

    ssize_t n = read(0, file_name, sizeof(file_name) - 1);

    if (n <= 0) return 1;

    file_name[n] = '\0';
    if (file_name[n - 1] == '\n') file_name[n - 1] = '\0';

    int file = open(file_name, O_RDONLY);
    if (file == -1) {
        const char* msg = "open file error\n";
        write(2, msg, strlen(msg));
        return 1;
    }

    char shm_name[64], sem_data_name[64], sem_result_name[64];
    pid_t pid = getpid();
    snprintf(shm_name, sizeof(shm_name), "/shm_%d", pid);
```

```
    snprintf(sem_data_name, sizeof(sem_data_name), "/sem_d_%d", pid);
    snprintf(sem_result_name, sizeof(sem_result_name), "/sem_r_%d", pid);

    int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(SharedData));
    SharedData* shared = (SharedData*)mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);

    sem_t* sem_data = sem_open(sem_data_name, O_CREAT, 0666, 0);
    sem_t* sem_result = sem_open(sem_result_name, O_CREAT, 0666, 0);

    char file_buf[8192];
    ssize_t r = read(file, file_buf, sizeof(file_buf) - 1);
    if (r > 0) file_buf[r] = '\0';
    close(file);

    shared->total_lines = 0;
    int num_index = 0;

    char* line = file_buf;
    while (line != nullptr && *line != '\0') {
        char* nl = strchr(line, '\n');
        if (nl != nullptr) *nl = '\0';

        int count = parse_line(line, &shared->input_nums[num_index]);
        if (count > 0) {
            shared->line_lengths[shared->total_lines] = count;
            num_index += count;
            shared->total_lines++;
        }

        if (nl == nullptr) break;
        line = nl + 1;
    }

    pid = fork();

    if (pid == -1) {
        const char* msg = "fork error\n";
        write(2, msg, strlen(msg));
        return 1;
    }

    if (pid == 0){

        sem_wait(sem_data);
        shared->error_flag = 0;

        int idx = 0;
        for (int l = 0; l < shared->total_lines; l++) {
            int len = shared->line_lengths[l];
            if (len == 0) continue;
            double result = shared->input_nums[idx];
            for (int i = 1; i < len; i++) {
                double x = shared->input_nums[idx + i];
                if (x == 0.0) {
                    shared->error_flag = 1;
                    sem_post(sem_result);
```

```
                exit(1);
            }
            result /= x;
        }
        shared->output_result[l] = result;
        idx += len;
    }
    sem_post(sem_result);
    exit(0);

} else {

    sem_post(sem_data);
    sem_wait(sem_result);

    int status;
    waitpid(pid, &status, 0);

    if (shared->error_flag == 1) {
        const char* msg = "division by zero\n";
        write(2, msg, strlen(msg));
    } else {

        for (int l = 0; l < shared->total_lines; l++) {
            char out[64];
            int len = snprintf(out, sizeof(out), "%.4f\n", shared->output_result[l]);
            write(1, out, len);
        }
    }

    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    shm_unlink(shm_name);

    sem_close(sem_data);
    sem_unlink(sem_data_name);

    sem_close(sem_result);
    sem_unlink(sem_result_name);

}

return 0;
}
```

Протокол работы программы

Тестирование:

input.txt
1.0500
9.0000
-5.0000
0.3333
2000.0000
-1.4000

f3.txt
division by zero

Вывод

В ходе работы были изучены механизмы межпроцессного взаимодействия через именованную разделяемую память (shared memory) и синхронизацию с помощью именованных семафоров. Была разработана программа, в которой родительский процесс подготавливает данные в общей памяти, а дочерний процесс выполняет вычисления и записывает результат обратно, используя семафоры для предотвращения состояния гонки и взаимных блокировок. Освоены системные вызовы shm_open, mmap, sem_open и их корректная очистка (unlink). Полученный результат подтверждает, что связка разделяемой памяти и семафоров является более быстрым способом обмена данными по сравнению с каналами (pipe), так как исключает накладные расходы на копирование данных через ядро.